

Multicore

Mattia Maiorano - 2009650
Relazione sequence

Introduzione

1.1] Il progetto

La scelta per la consegna è stata **sequence**. L'analisi di una stringa di DNA alla ricerca di matching fra quest'ultima e i pattern.

Il codice si articolava nella generazione della sequenza, delle sottosequenze, delle allocazioni di memoria e infine nel ciclo di matching.

Alcune delle difficoltà implementative riguardavano l'accesso simultaneo ad aree di memoria condivise, nel particolare:

- **Pat_matches**, integer incaricato di tenere conto del numero di successi
- **Pat_found**, array contenente la casella di ritrovamento del pattern, al numero di indice pari al pattern
- **Seq_matches**, array contenente il numero di volte quell'indice nella sequenza sia stato riscontrato in un pattern (chiamato in `increment_matches`)

E ancora alcuni problemi riguardanti l'utilizzo di variabili private per lo scorrimento dei due for interni: **start** e **lind**.

```
/* 5. Search for each pattern */
unsigned long start;
int pat;
for( pat=0; pat < pat_number; pat++ ) {

    /* 5.1. For each possible starting position */
    for( start=0; start <= seq_length - pat_length[pat]; start++ ) {

        /* 5.1.1. For each pattern element */
        for( lind=0; lind<pat_length[pat]; lind++ ) {
            /* Stop this test when different nucleotids are found */
            if ( sequence[start + lind] != pattern[pat][lind] ) break;
        }
        /* 5.1.2. Check if the loop ended with a match */
        if ( lind == pat_length[pat] ) {
            pat_matches++;
            pat_found[pat] = start;
            break;
        }
    }

    /* 5.2. Pattern found */
    if ( pat_found[pat] != (unsigned long)NOT_FOUND ) {
        /* 4.2.1. Increment the number of pattern matches on the sequence positions */
        increment_matches( pat, pat_found, pat_length, seq_matches );
    }
}
```

1.2] L'idea di parallelizzazione

Sono saltate subito all'occhio due possibili vie: la suddivisione della sequenza principale di DNA oppure la suddivisione del complesso di sottosequenze.

La **prima soluzione** si presentava, come l'altra, molto intuitiva, ma si è subito notato come suddividere la sequenza potesse generare problemi:

ACCGTCAG => ACCG - TCAG non da riscontro positivo per GTC

Si è dunque ragionato su un modo per tamponare questo fenomeno e la sovrapposizione di una sezione della stringa alla successiva scongiurava effettivamente i troncamenti, in cambio però di lunghezza aggiuntiva:

ACCGTCAG => ACCGTC - TCAG da riscontro positivo per GTC

Questo metodo porta però una grande soggezione al rapporto di lunghezza tra sequenza e sottosequenze.

La nuova sequenza complessiva avrà infatti, assumendo **n** come lunghezza della sequenza iniziale, **m** come lunghezza massima delle sottosequenze e **p** numero dei processi:

$$N = (n / p) + (n / p + m - 1) * (p - 1)$$

Alla luce di questo si è scelto di scartare questa strada.

La seconda soluzione si è presentata fin da subito più snella e libera da dipendenze computazionali con l'input.

Si è notato il suo costo crescesse in maniera lineare con l'aumento delle sottosequenze ed è stato un motivo più che sufficiente per la sua scelta.

1.3] Le librerie

Per la soluzione singola si è voluto optare per **openMP**.

Si è pensato potesse offrire una tipologia di lavoro diverso rispetto a quello che sarebbe stato svolto per MPI e CUDA, con una libreria più progettuale.

Per la soluzione combinata si è voluto optare per **MPI+CUDA**.

Due librerie molto estese e molto potenti che è necessario saper gestire assieme al fine di trarre il meglio della potenza delle GPU.

Sono state scelte anche per presentare invece una sfida più tecnica rispetto openMP.

OpenMP

2.1] Prima implementazione

OpenMP fornisce fra i suoi vari pragmas **parallel for**, questo sembrava calzare perfettamente sul grande for al centro del nostro codice.

Sono stati scelti come input per i test i tre test suggeriti nel README e due versioni potenziate rispettivamente su lunghezza sequenza e numero pattern:

1. 300 0.1 0.3 0.35 100 5 5 300 150 50 150 80 M 609823
2. 1000 0.35 0.2 0.25 0 0 0 20000 10 0 500 0 M 4353435
3. 10000 0.35 0.2 0.25 0 0 0 10000 9000 9000 50 100 M 4353435
4. 100000 0.35 0.2 0.25 0 0 0 10000 9000 9000 50 100 M 4353435
5. 10000 0.35 0.2 0.25 0 0 0 100000 9000 9000 50 100 M 4353435

Ogni input ha avuto un suo scopo:

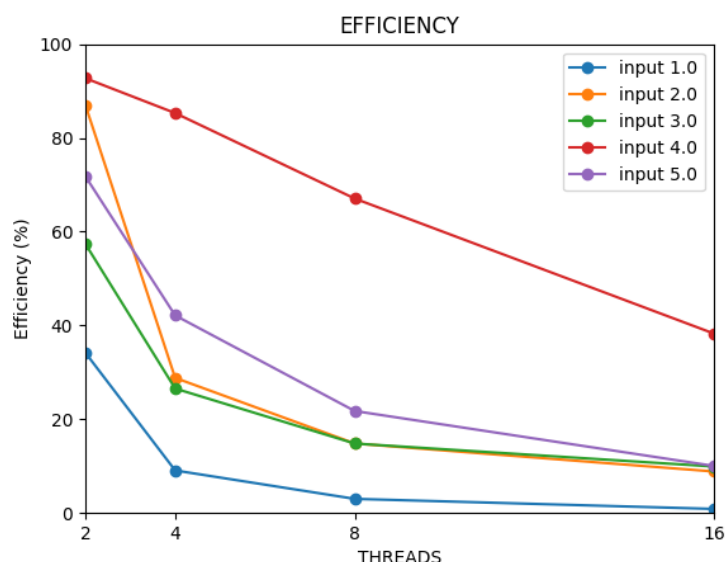
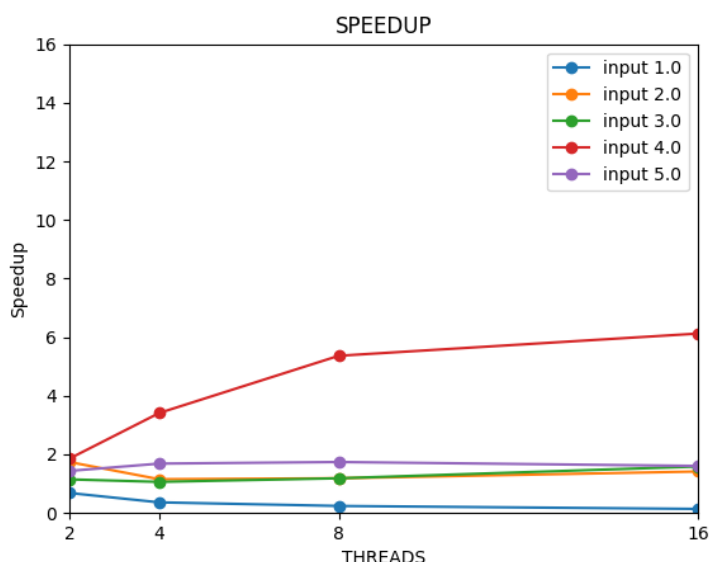
- Il **primo** e il **secondo** sono stati utili per misurare l'overhead
- Il **terzo** controllava eventuali difficoltà dovute alla concorrenza (poiché i pattern generati erano tutti appartenenti alla lista)
- Il **quarto** e il **quinto** evidenziavano se il codice fosse più soggetto a rallentamenti dovuti ad aumento di pattern o lunghezza della sequenza

2.2.1] Trial end Error - primo test

Il primo tentativo è stato molto basilare, si sono risolti i problemi di variabili condivise fra i thread attraverso l'utilizzo di sezioni critiche.

Volontariamente non sono state applicate clause di **scheduling** o **reduction** al fine di creare una base da cui partire, seguono i test.

```
/* 5.1.2. Check if the loop ended with a match */
if ( lind == pat_length[pat] ) {
    #pragma omp atomic
    pat_matches++;
    #pragma omp atomic write
    pat_found[pat] = start + 1;
    #pragma omp critical
    {
        increment_matches( pat, pat_found, pat_length, seq_matches );
    }
    break;
}
```



Come possiamo vedere ci sono stati grossi limitamenti sia dal punto di vista dello **speedup** che soprattutto dell'**efficienza**.

Era un esito sicuramente facile da immaginare visto l'alto uso di sezioni critiche, probabile causa di un collo di bottiglia.

A conferma di questo possiamo vedere come l'unico test con risultati leggermente migliori sia stato il test **4**, con rapporto **sequenza di DNA/numero di pattern** maggiore.

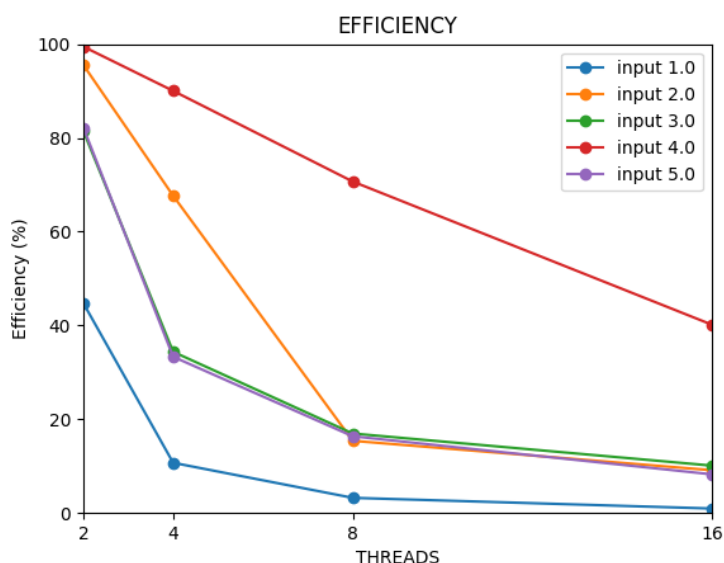
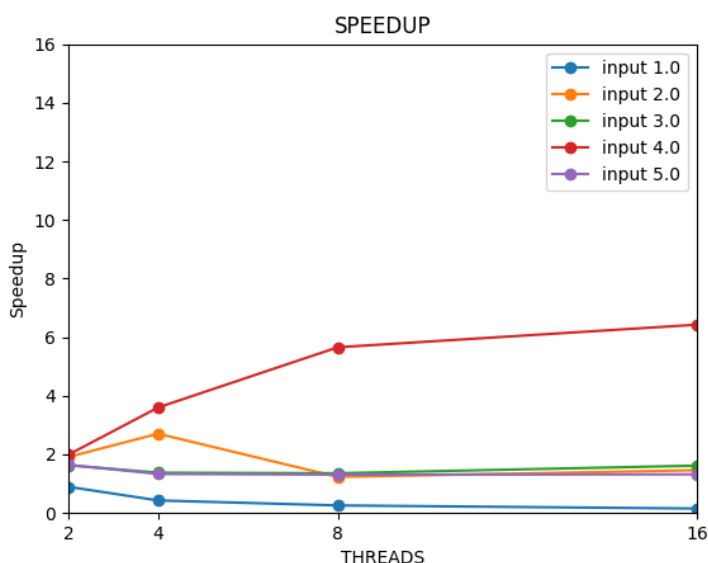
2.2.2] Trial end Error - secondo test

Nel secondo tentativo si è adottato l'utilizzo delle **operazioni atomiche**, sia nell'incremento di pat_matches che nell'update di pat_found, al fine di ridurre i bottlenecks.

È stata applicata una sezione critica ad increment_matches poiché anche al suo interno erano presenti accessi di memoria pericolosi e dunque troppe chiamate atomiche.

Prima di procedere si è presa in considerazione la possibile non mutua esclusione fra operazioni atomiche e sezioni critiche, che avrebbe potuto danneggiare la lettura di pat_found internamente a increment_matches, ma empiricamente si è visto come ciò non avvenisse.

I test.



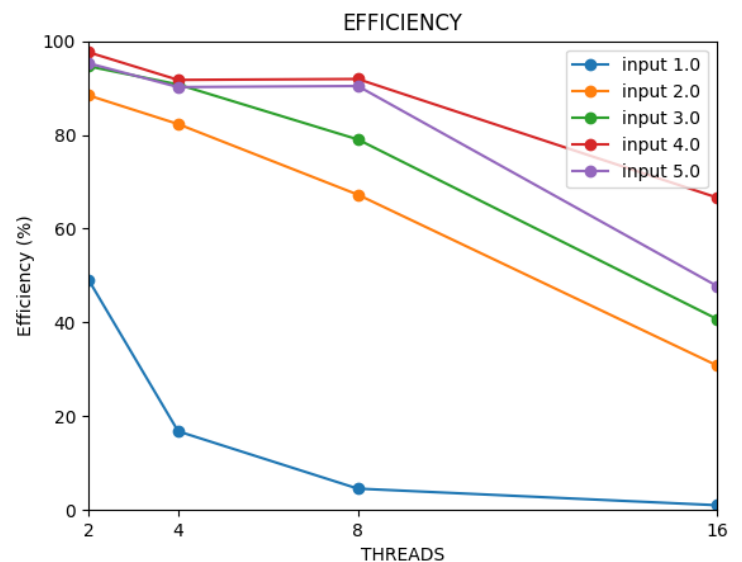
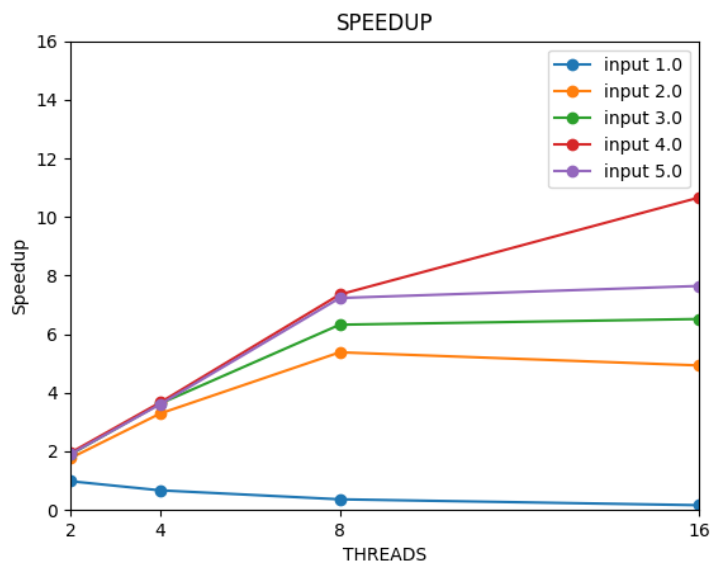
È interessante notare come troviamo di nuovo la prestazione migliore nel test **4** con la sequenza di DNA più lunga, e come i test **3** e **5**, rispettivamente 10.000 e 100.000 pattern, ottengano risultati molto simili.

Il fatto che, nonostante sia il test **3** che **4** utilizzino 10.000 pattern, il test **3** sia più simile al test **5** in quanto a risultati, **sembra poter concludere che il vero discriminante sia la lunghezza del DNA.**

2.2.3] Trial end Error - terzo test

I risultati sono ancora poco soddisfacenti ed è chiaro sia necessario trovare una soluzione diversa che aggiri i colli di bottiglia (per operazioni atomiche o sezioni critiche) dovuti all'alto numero di riscontri e conseguenti scritture.

Si è voluto dunque cambiare l'approccio a favore di tre clausole di **riduzione** su pat_matches, pat_found e seq_matches.



L'idea è stata molto buona.

Ad eccezione del primo test infatti, decisamente troppo piccolo, le reduction hanno portato ad un'efficienza estremamente più alta.

Sostenuta meglio in particolare dal test a sequenza più lunga.

2.2.4] Trial end Error - test finali di ottimizzazione

Con l'idea di non lasciare nulla al caso, sono stati eseguiti test con tecniche secondarie che avrebbero potuto spingere ancora un po' più in alto le prestazioni, come ad esempio **collapse**.

Sono stati però riscontrati vari problemi dovuti alla natura poco adatta all'unione dei due for (non è stato infatti possibile utilizzare la clausola collapse di openMP), e questa difficoltà nell'unione si è vista a livello prestazionale.

Altre soluzioni, come **dynamic** o **guided scheduling**, sono state visionate rapidamente.

Essendo a livello logico poco adatte (dynamic ad esempio non avrebbe apportato vantaggi visto la lunghezza fissa dei pattern in questi test) e non essendo supportate dai test non sono state approfondite.

2.3] Conclusioni

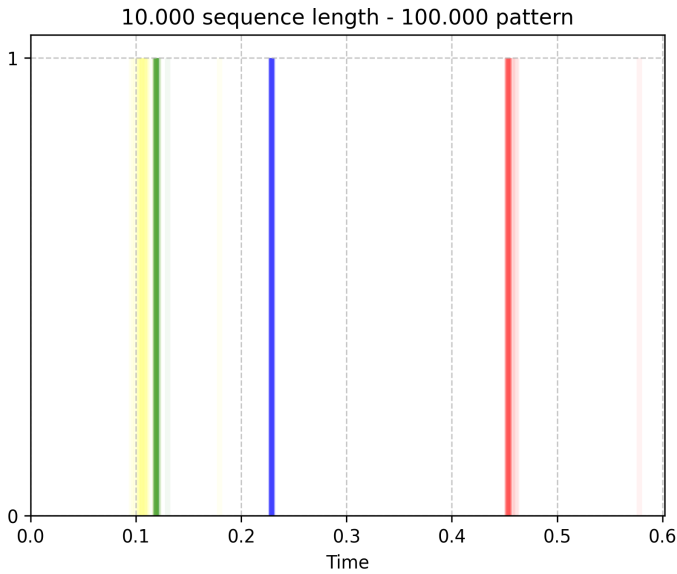
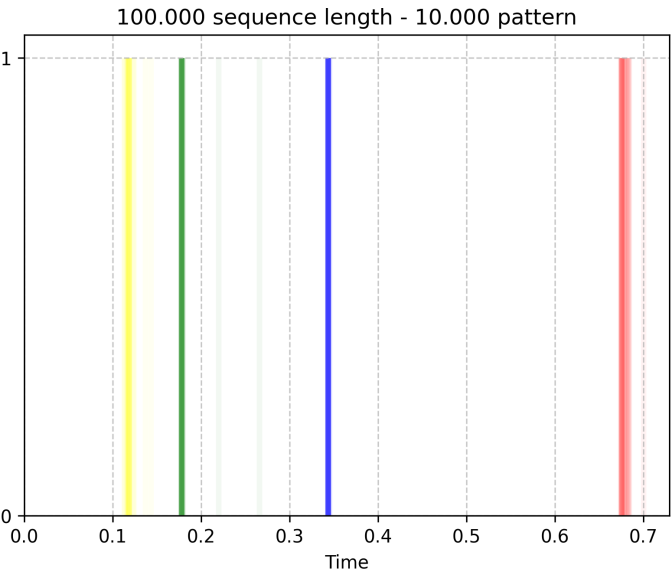
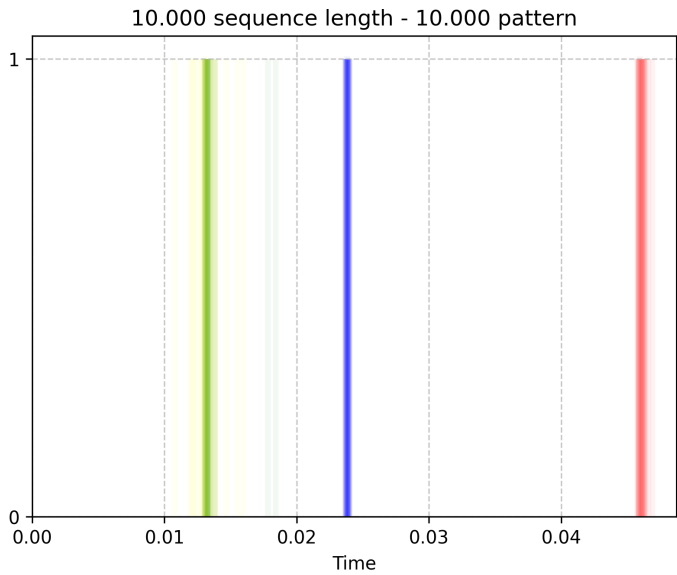
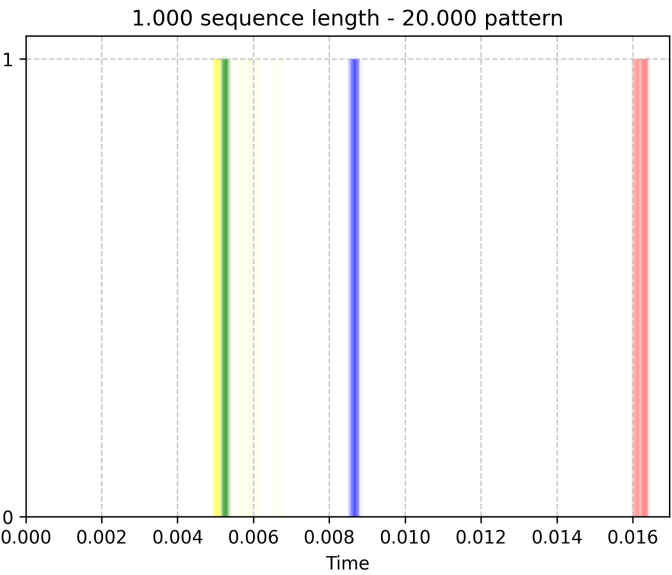
La soluzione più vantaggiosa è risultata essere l'utilizzo delle reduction, che hanno eliminato i colli di bottiglia dovuti agli elevati accessi contemporanei.

È doveroso però considerare come i test implementassero situazioni ad **alta concorrenza** dovute all'utilizzo esclusivo di pattern appartenenti alla sequenza totale e come dunque in situazioni più generiche questo ostacolo sarebbe potuto presentarsi in forma minore.

Questo avrebbe reso più valide anche le soluzioni con le sezioni critiche le quali, anche se non con gli stessi risultati, avrebbero apportato un notevole **risparmio di memoria** rispetto le due reduction su array di notevoli dimensioni.

Un'analisi ulteriore possiamo farla sulla **varianza** dei risultati ottenuti e su quanto il **rumore** possa generare problemi e ritardi nei risultati.

Di seguito riporto la distribuzione dei risultati nel miglior test in quanto a tempo, con
rispettivamente: 2, 4, 8 e 16 core.



3.1] Prima implementazione

Come per openMP si è voluto partire da un'idea semplice su cui costruire, con un kernel e una implementazione MPI di base.

Si è suddivisa la totalità dei pattern fra i vari **thread** (oltre che fra i **processi**), con un seguente calcolo per blocchi e thread ognuno da incaricare alla ricerca di un pattern (con un problema centrale di mutua esclusione).

```
/* ADD KERNELS AND OTHER FUNCTIONS HERE */
__global__ void search_patterns(char *d_sequence, char **d_pattern, unsigned long *d_pat_le
    int pat = blockIdx.x * blockDim.x + threadIdx.x;
    if (pat >= pat_number) return;

    unsigned long start, lind;

    for (start = 0; start <= seq_length - d_pat_length[pat]; start++) {

        for (lind = 0; lind < d_pat_length[pat]; lind++) {
            if (d_sequence[start + lind] != d_pattern[pat][lind]) break;
        }
        if (lind == d_pat_length[pat]) {
            atomicAdd(&d_pat_matches, 1);
            d_pat_found[pat] = start + 1;
            for (lind = 0; lind < d_pat_length[pat]; lind++) {
                atomicAdd(&d_seq_matches[start + lind], 1);
            }
            break;
        }
    }
}
```

Per i **test** sono stati scelti degli input diversi più adatti al carico di lavoro di una GPU, di seguito:

1. 100000 0.35 0.2 0.25 0 0 0 100000 9000 9000 50 100 M 4353435
2. 10000 0.35 0.2 0.25 0 0 0 1000000 9000 9000 50 100 M 4353435
3. 1000000 0.35 0.2 0.25 0 0 0 10000 9000 9000 50 100 M 4353435

Cominciamo analizzando i risultati ottenuti da questa prima serie di test fra sequenziale e codice parallelo CUDA+MPI ad un processo:

SEQUENZIALE	CUDA + MPI	SPEEDUP
13,4622 sec	0,2437 sec	55,24
9,1255 sec	0,4176 sec	21,85
14,0797 sec	0,5784 sec	24,34

Da notare che non è stato possibile una rappresentazione come sopra per la limitazione dei nodi con GPU, si propone dunque una notazione più compatta ma comunque risultato di media/mediana dei vari test

Possiamo notare dei risultati promettenti, con un lieve vantaggio per l'esecuzione più equilibrata a sfavore dei test a più alto numero di pattern, sicuramente più soggetti a bottlenecks, e di lunghezza di sequenza, probabilmente per un fattore di trasferimenti di memoria e alto tasso di **swap**.

Nei vari test si è voluto sperimentare anche con il numero di thread da generare per blocco e si è individuato **256** come compromesso migliore fra **limitazione di parallelismo** e **latency hiding**.

3.2.1] CUDA - memory usage

Questo è il **kernel** attuale, come si può osservare molto snello e privo di sezioni critiche a favore delle operazioni atomiche.

```
__global__ void search_patterns(char *d_sequence, char **d_pattern, unsigned long *d_pat_length,
    int pat = blockIdx.x * blockDim.x + threadIdx.x;
    if (pat >= pat_number) return;

    unsigned long start, lind;

    for (start = 0; start <= seq_length - d_pat_length[pat]; start++) {

        for (lind = 0; lind < d_pat_length[pat]; lind++) {
            if (d_sequence[start + lind] != d_pattern[pat][lind]) break;
        }
        if (lind == d_pat_length[pat]) {
            atomicAdd(&d_pat_matches, 1);
            d_pat_found[pat] = start + 1;
            for (lind = 0; lind < d_pat_length[pat]; lind++) {
                atomicAdd(&d_seq_matches[start + lind], 1);
            }
            break;
        }
    }
}
```

Un primo tentativo svolto è stato quello di implementare l'uso della **pinned memory** per velocizzare il trasferimento di memoria che, con la grandezza dei test, è parecchio rilevante.

```
char *d_sequence;

CUDA_CHECK_FUNCTION( cudaHostRegister(sequence, sizeof(char) * seq_length, cudaHostRegisterDefault) );

CUDA_CHECK_FUNCTION( cudaMalloc( &d_sequence, sizeof(char) * seq_length ) );
CUDA_CHECK_FUNCTION( cudaMemcpy( d_sequence, sequence, sizeof(char) * seq_length, cudaMemcpyHostToDevice ) );

CUDA_CHECK_FUNCTION( cudaHostUnregister(sequence) );
```

Si è avviato il codice ad un solo processo al fine di escludere il più possibile MPI e si sono registrati dei piccoli miglioramenti su tutti e tre i test:

SEQUENZIALE	CUDA + MPI	SPEEDUP	MIGLIORAMENTO
13,4622 sec	0,2317 sec	58,10	5%
9,1255 sec	0,3780 sec	24,14	10,5%
14,0797 sec	0,5402 sec	26,06	7%

Notiamo un incremento maggiore generale e soprattutto nei casi a maggior numero di pattern.

I test ad alto numero di pattern potrebbero averne giovato di più rispetto ai sequence-oriented poiché nel complesso più pesanti (essendo unsigned long più grande di byte).

Sono stati poi svolti alcuni tentativi di breve durata su:

- **Disabilitazione cache L1**
- **Textures**

- **Read-only memory**

Per quanto riguarda la disabilitazione della cache e le textures i risultati sono stati sensibilmente peggiori.

Nel primo caso probabilmente a causa dell'alto tasso di lettura piuttosto che scrittura e nel secondo più semplicemente a causa di inquinamento della cache a sfavore di altri elementi.

Per quanto riguarda la read-only memory le aspettative invece erano più alte specialmente per la natura sequenziale degli accessi, ma anche qui non sono stati riscontrati miglioramenti.

In entrambi un grande ostacolo è stato **l'alto volume degli input** che ha limitato lo sfruttamento della cache.

Per lo stesso motivo si è esclusa la dichiarazione di variabili in shared memory riguardanti direttamente i due valori di input, in quanto avrebbero ecceduto lo spazio.

3.2.2] CUDA - reductions

Non potendo sfruttare a pieno la cache si è dunque deciso di passare all'ottimizzazione dei vari colli di bottiglia interni al kernel.

Il primo preso in esame è stata la chiamata atomica **addAtomic** su pat_matches.

Di nuovo la misura elevata dell'input ci mette in difficoltà sulle implementazioni, la riduzione è stata infatti costruita attraverso altri due kernel d'appoggio:

```
__global__ void partial_reduce(int *d_block_pat_matches, int *d_total_matches, int length) {
    extern __shared__ int all_matches[];

    int tid = threadIdx.x;
    int global_idx = blockIdx.x * blockDim.x + tid;
    int blockid = blockIdx.x;
    int upper_limit = min(1024, length - blockDim.x * blockIdx.x);

    if (global_idx < length) {
        all_matches[tid] = d_block_pat_matches[global_idx];
    } else {
        all_matches[tid] = 0; // Initialize unused shared memory to avoid undefined behavior
    }

    __syncthreads();

    if (global_idx >= length) return;

    // Perform binary tree reduction
    int red_length = upper_limit;
    for (int stride = (red_length + 1) / 2; stride > 0; stride = (stride + 1) / 2) {
        __syncthreads();
        if (tid + stride < red_length) {
            all_matches[tid] += all_matches[tid + stride];
        }

        red_length = stride;
        if (stride == 1) {
            break;
        }
    }
    __syncthreads();

    if (threadIdx.x == 0) {
        d_total_matches[blockid] = all_matches[0];
    }
}
```

```
__global__ void reduced_sum(int *d_block_pat_matches, int *d_total_matches, int length) {
    extern __shared__ int shared_data[];

    int tid = threadIdx.x;
    int global_idx = blockIdx.x * blockDim.x + tid;

    if (global_idx < length) {
        shared_data[tid] = d_block_pat_matches[global_idx];
    } else {
        shared_data[tid] = 0; // Initialize unused shared memory to avoid undefined behavior
    }

    __syncthreads();

    // Perform binary tree reduction
    int red_length = blockDim.x;
    for (int stride = (red_length + 1) / 2; stride > 0; stride = (stride + 1) / 2) {

        if (tid + stride < red_length) {
            shared_data[tid] += shared_data[tid + stride];
        }

        __syncthreads();
        red_length = stride;
        if (stride == 1) {
            break;
        }
    }

    // Write the result from thread 0 to global memory
    if (tid == 0) {
        d_total_matches[0] = shared_data[0];
    }
}
```

SEQUENZIALE	CUDA + MPI	SPEEDUP	MIGLIORAMENTO
13,4622 sec	0,4433 sec	30,37	-48%
9,1255 sec	0,4730 sec	19,29	-21%
14,0797 sec	1,7482 sec	8,054	-69,8%

Una prima implementazione con shared memory e riduzione esclusivamente interna al primo kernel era stata tentata, ma non apportava miglione

Purtroppo notiamo un peggioramento generale, soprattutto in quei test con il rapporto **seq_length / pat_num** maggiore.

Il motivo potrebbe essere in realtà la poca concorrenza generata, in quanto un thread si troverà a chiamare l'operazione atomica solo una volta durante la sua esecuzione.

Il codice è stato dunque riportato a prima delle modifiche e l'attenzione è passata ad un'altra operazione atomica: l'incremento di seq_matches.

Questi i risultati:

SEQUENZIALE	CUDA + MPI	SPEEDUP	MIGLIORAMENTO
13,4622 sec	0,3106 sec	43,34	-25%
9,1255 sec	0,4520 sec	20,19	-16%
14,0797 sec	0,6119 sec	23,01	-12%

Come si può vedere i test continuano a essere peggiori rispetto al miglior test precedente, c'è quindi da ragionare sulle motivazioni.

3.2.3] CUDA - considerazioni

Si sono incontrati problemi sia nell'implementazione di soluzioni a memoria condivisa che di riduzioni e, a fronte dei dati raccolti, a parte delle piccole accortezze, non è stato possibile apportare grandi miglioramenti all'implementazione di partenza.

Le modifiche alla **gestione di memoria** si sono spesso scontrate con le grandi dimensioni dell'input, che sfruttava già a pieno la cache a fronte delle moltissime operazioni di lettura.

Le modifiche alle **operazioni di scrittura** (le riduzioni) hanno invece perso il confronto con le più leggere e rapide operazioni atomiche, libere dal grande overhead necessario a reduction su così grandi quantità di dati.

Con overhead ci si riferisce ai molteplici lanci di kernel e allocazioni di memoria necessarie allo smaltimento di array locali e simili.

La vicinanza delle soluzioni dei pattern ha fatto sì che anche la stessa redistribuzione del lavoro nell'opera di riduzione finale si aggregasse in maniera maggiore su alcuni blocchi piuttosto che altri.

Portando dunque a scegliere se favorire il **coalescing** o un'equa distribuzione del **carico computazionale**.

Ciò che si può raccogliere da questa analisi è come l'utilizzo di un così denso set in input, sia sotto il punto di vista di dimensione che di concorrenza, abbia estremamente limitato lo spazio di movimento.

3.3.1] MPI - prima implementazione

Dopo aver lavorato su CUDA si è voluto prendere il corpo del codice e cercare di sfruttarne la potenza su più GPU attraverso MPI, con un approccio di suddivisione **estremamente** simile a quello già testato e descritto sopra per openMP.

Di seguito i risultati.

SEQUENZIALE	CUDA + MPI (1 proc.)	CUDA + MPI (2 proc.)	SPEEDUP
12,3286 sec	0,5433 sec	0,2365 sec	2,30
8,0129 sec	0,9142 sec	0,4320 sec	2,12
13,1786 sec	0,7130 sec	0,4941 sec	1,44

Da notare che i test per MPI sono stati svolti sulla piattaforma Kaggle, motivo per cui potrebbero differire in parte dai precedenti, e che, a causa delle limitazioni sia di Kaggle che del cluster, i test purtroppo sono stati svolti solo da 1 a 2 nodi.

È molto interessante notare come la suddivisione del carico di lavoro oltre che dimezzare il tempo abbia probabilmente a volte **smaltito i colli di bottiglia** rendendo più efficiente il lavoro delle GPU (da cui lo speedup maggiore a 2).

3.3.2] MPI - ritocchi

Dopo l'ottimo risultato si è voluto limare l'efficienza di MPI nei momenti più critici come le reduce, soprattutto visto il grande numero di elementi da processare.

In particolare si è vista un'occasione di messa a punto nell'utilizzo di **MPI_Gather**, in media più lento rispetto ad **MPI_Reduce**.

Per seq_matches infatti si è potuto re-implementare il Gather come Reduce grazie ad **MPI_SUM**, la quale ricomponeva i risultati parziali dei vari processi.

*I risultati ottenuti dai vari processi su seq_matches sono infatti sparsi su tutto l'array e in realtà la sua implementazione più naturale era **già** una reduce.*

La questione di pat_found è stata però più complessa poiché pat_found è un array di **unsigned long**, non inizializzati a 0 ma al massimo valore registrabile (unsigned long) - 1).

La soluzione si è concretizzata in una modifica all'operazione di lettura.

L'array è stato infatti inizializzato a 0 e tutti gli indici conservati slittati di 1, rendendo quindi il vettore papabile per una reduce a patto che in lettura il valore venisse ridotto di 1.

Si può ottenere un risultato simile con MPI_MIN, ma essendo MPI_SUM solitamente più ottimizzata si è optato per quest'ultima.

SEQUENZIALE	CUDA + MPI (1 proc.)	CUDA + MPI (2 proc.)	SPEEDUP
12,3286 sec	0,5528 sec	0,2513 sec	2,20
8,0129 sec	0,9593 sec	0,4305 sec	2,23
13,1786 sec	0,7629 sec	0,4962 sec	1,54

Lo speedup risulta incrementato anche se la differenza sulle esecuzioni a doppio processo sembra minima, poiché si è effettuata una rimisurazione anche a singolo processo e si è notato un peggioramento generale delle prestazioni di Kaggle al momento delle misurazioni, che però sembrano confermare un trend positivo.

3.4] Conclusioni

Nonostante alcune difficoltà nello spingere CUDA ai suoi migliori risultati, la collaborazione con MPI ne ha esaltato le qualità, ha infatti permesso di snellire il carico di lavoro e portare ad un'**efficienza molto elevata**, la quale è immaginabile possa addirittura accrescere con l'aumento dei processi.